

JavaScript Testing — Senior Developer Level

Testing-এর মূল উদ্দেশ্য শুধু:

“Code কাজ করছে কিনা?”

তা না।

ভালো testing-এর উদ্দেশ্য:

Correctness
+
Regression Protection
+
Design Feedback
+
Refactoring Confidence
+
Documentation of Behavior

একজন junior developer সাধারণত ভাবে:

Code লিখলাম
↓
Browser-এ চাললাম
↓
কাজ করছে

একজন senior developer ভাবে:

Code
↓
Expected behavior কী?
↓
কোন কোন edge case আছে?
↓
কীভাবে automatically verify করব?
↓
Future change করলে কীভাবে regression আটকাব?

Testing-এর Basic Mental Model

একটি function:

```
function add(a, b) {  
  return a + b;  
}
```

Expected behavior:

```
add(2, 3)  
  ↓  
5
```

Test:

```
expect(add(2, 3)).toBe(5);
```

Testing basically:

```
Input  
  ↓  
System Under Test  
  ↓  
Output  
  ↓  
Expected Result-এর সাথে Compare
```

93. Unit Testing Mindset — Pure Functions First

Unit Test কী?

একটি ছোট, isolated unit-এর behavior test করাকে সাধারণভাবে unit testing বলা হয়।

Unit হতে পারে:

```
Function  
Class method
```

Utility
Small module

সবচেয়ে সহজ unit:

```
function multiply(a, b) {  
  return a * b;  
}
```

Test:

```
expect(multiply(2, 5)).toBe(10);
```

Pure Function দিয়ে শুরু করা কেন ভালো?

Pure function:

Same input
↓
Same output

এবং external state modify করে না।

Example:

```
function calculateTotal(price, quantity) {  
  return price * quantity;  
}
```

এখানে:

calculateTotal(100, 2)
↓
200

প্রতিবার একই result।

Pure Function-এর সুবিধা

Pure function:

- Easy to test
- Easy to reason about
- Easy to debug
- Easy to reuse
- Easy to refactor

তাই testing শেখার সময় প্রথমে:

Pure function
↓
Simple unit test

দিয়ে শুরু করা সবচেয়ে ভালো।

Impure Function

ধরো:

```
let total = 0;  
  
function addToTotal(value) {  
  total += value;  
}
```

এখানে output global state-এর ওপর depend করছে।

Previous state
↓
addToTotal()
↓
New state

Test setup/cleanup বেশি complex হতে পারে।

External Dependency

আরও complex:

```
async function getUser(id) {  
  const response = await fetch(`/users/${id}`);  
  return response.json();  
}
```

এখানে:

```
getUser()  
  ↓  
Network  
  ↓  
External system
```

এটা pure function না।

এখানে unit testing-এর জন্য dependency isolate করার প্রয়োজন হতে পারে।

Unit Test-এর AAA Pattern

একটি খুব useful pattern:

Arrange → Act → Assert

Example:

```
test("calculates total", () => {  
  // Arrange  
  const price = 100;  
  const quantity = 2;  
  
  // Act  
  const result = calculateTotal(price, quantity);  
  
  // Assert  
  expect(result).toBe(200);  
});
```

Arrange

Test-এর জন্য setup ।

Act

যেটা test করছে সেটা execute ।

Assert

Expected result verify ।

Mental model:

```
Arrange
  ↓
Act
  ↓
Assert
```

Good Unit Test কেমন?

ভালো unit test সাধারণত:

```
Fast
Deterministic
Isolated
Readable
Focused
```

একটি test ideally একটি behavior verify করবে ।

Bad:

```
এক test - এর মধ্যে
10টা unrelated behavior
```

Better:

```
test A → login success  
test B → invalid password  
test C → locked account
```

Edge Cases

শুধু happy path test করলে হবে না।

Function:

```
function divide(a, b) {  
  if (b === 0) {  
    throw new Error("Cannot divide by zero");  
  }  
  
  return a / b;  
}
```

Tests:

```
2 / 1  
10 / 2  
0 / 5  
negative values  
division by zero
```

Testing mindset:

```
Happy Path  
+  
Boundary  
+  
Invalid Input  
+  
Error Path
```

94. Mocking — Stubs, Spies, Mocks Differences

Mocking বুঝতে হলে আগে **test doubles** বুঝতে হবে।

ধরো তোমার code:

```
async function createUser(user) {  
  await database.save(user);  
}
```

Test করার সময় real database ব্যবহার করতে চাও না।

তাহলে database-এর বদলে একটা substitute object দিতে পারো।

এটাই broadly **test double**।

Test Double

```
Real Dependency  
  ↓  
  Replace  
  ↓  
Test Double
```

Common types:

```
Dummy  
Stub  
Spy  
Mock  
Fake
```

এগুলো অনেক সময় loosely “mock” বলে ডাকা হয়, কিন্তু technically আলাদা concept।

Stub

Stub-এর কাজ:

Predefined answer দেওয়া।

ধরো:

```
const userRepository = {  
  findById: async () => ({  
    id: 1,  
    name: "Rahim"  
  })  
};
```

এখানে repository real database-এ যাচ্ছে না।

এটা predefined data return করছে।

```
Call  
↓  
Stub  
↓  
Fixed Result
```

Stub Example

```
const paymentService = {  
  getBalance: async () => 1000  
};  
  
async function canBuy() {  
  const balance = await paymentService.getBalance();  
  
  return balance >= 500;  
}
```

Test-এর সময়:

```
getBalance()  
↓  
Stub
```

↓
1000

তারপর verify:

canBuy()
↓
true

Spy

Spy-এর উদ্দেশ্য:

Dependency কীভাবে ব্যবহার হয়েছে সেটা **observe** করা।

যেমন:

```
const emailService = {  
  send() {}  
};
```

Spy দিয়ে check করতে পারো:

send() কি call হয়েছে?
কতবার?
কী arguments দিয়ে?

Example conceptual:

```
expect(emailService.send).toHaveBeenCalledWith(  
  "user@example.com"  
);
```

Stub বনাম Spy

Stub	Spy
Result/control দেয়	Behavior observe করে
“কী return করবে?”	“কীভাবে call হয়েছে?”
State control	Interaction verification

সহজভাবে:

Stub → Control output

Spy → Observe interaction

Mock

Mock শব্দটি দুইভাবে ব্যবহৃত হয়।

Broadly test double বোঝাতেও “mock” বলা হয়।

কিন্তু strict testing terminology-তে mock হলো এমন test double যেখানে expected interaction আগে থেকেই define করা হয়।

যেমন:

Expected:
database.save(user)
exactly once

Test fail করবে যদি expected interaction না হয়।

Concept:

Expected Interaction
↓
Mock
↓
Was expectation satisfied?

Mock vs Spy

Spy:

Call হয়েছে?
Arguments কী?

Mock:

এই exact interaction হওয়া উচিত ছিল

অর্থাৎ mock বেশি expectation-driven।

95. Property-Based Testing Concept

Normal testing:

```
test("adds two numbers", () => {  
  expect(add(2, 3)).toBe(5);  
});
```

এখানে আমরা কয়েকটি specific input লিখি।

Property-based testing অন্যভাবে চিন্তা করে।

Specific example-এর বদলে input-এর **general property/invariant** define করা হয়।

Example

ধরো:

```
function reverseString(str) {  
  return str.split("").reverse().join("");  
}
```

একটি property:

```
reverse(reverse(x)) === x
```

যেকোনো valid string-এর জন্য।

```
x
↓
reverse(x)
↓
reverse(reverse(x))
↓
x
```

এটাই property।

Normal Testing vs Property-Based Testing

Normal:

```
"hello" → "olleh"
"abc"   → "cba"
"cat"   → "tac"
```

Property-based:

```
For every valid string x:
reverse(reverse(x)) === x
```

এতে testing system অনেক random input generate করতে পারে।

আরেকটি Example

ধরো:

```
function sortNumbers(numbers) {  
  return [...numbers].sort((a, b) => a - b);  
}
```

Properties:

Property 1

Output sorted হবে।

```
output[i] <= output[i + 1]
```

Property 2

Length same থাকবে।

```
input.length === output.length
```

Property 3

Original array mutate হবে না।

```
input unchanged
```

Property-Based Testing কেন powerful?

কারণ developer সব edge case manually ভাবতে পারে না।

Random generation:

```
Empty array  
Huge array  
Negative numbers  
Duplicates  
Very large numbers  
Random strings  
Unicode
```

এগুলো automatically generate হতে পারে।

Property vs Example

Example:

$$2 + 3 = 5$$

Property:

$$a + b = b + a$$

যদি addition-এর context ধরা হয়।

অর্থাৎ:

Example
↓
One case

Property
↓
General rule

Property-Based Testing কোথায় useful?

বিশেষ করে:

Parsers
Serialization
Data transformations
Sorting
Encoding/decoding
Mathematical functions
Utilities
State transformations

96. TDD — Red / Green / Refactor

TDD:

Test-Driven Development

এখানে code লেখার আগে test/expected behavior define করা হয়।

Classic cycle:

```
RED
↓
GREEN
↓
REFACTOR
↓
RED
↓
GREEN
↓
...
```

RED

প্রথমে এমন test লেখো যেটা fail করবে।

ধরো requirement:

```
calculateDiscount(100, 10) → 90
```

Test:

```
test("applies discount", () => {
  expect(calculateDiscount(100, 10)).toBe(90);
});
```

Function এখনো নেই।

Test:

FAIL

এটাই Red ।

GREEN

এখন minimum code লেখো যাতে test pass করে ।

```
function calculateDiscount(price, discount) {  
  return price - (price * discount) / 100;  
}
```

Test:

PASS

এটাই Green ।

REFACTOR

এখন code improve করো কিন্তু behavior change করবে না ।

```
Tests  
↓  
Safety net  
↓  
Refactor
```

Refactor-এর পর:

Tests still pass

TDD-এর মূল principle

```
Fail
↓
Minimum implementation
↓
Pass
↓
Improve design
```

TDD কেন useful?

TDD:

- Requirement clarify করে
- Small steps encourage করে
- Regression protection দেয়
- Design improve করতে সাহায্য করে
- Overengineering কমাতে পারে

TDD মানে কি সব code আগে test?

না।

TDD একটি development workflow।

Testing আর TDD এক জিনিস নয়।

```
Testing
↓
Tests লেখা/চালানো

TDD
↓
Test-first development process
```

TDD-এর common mistake

Developer কখনো এমন test লিখে:

Implementation-এর exact internal steps

তখন implementation সামান্য change করলেই test break করে।

Better:

Behavior test করো, **implementation detail** নয়।

97. Integration Testing Strategies

Unit test:

Function
↓
Isolated

Integration test:

একাধিক component/dependency একসাথে ঠিকভাবে কাজ করছে কিনা test করা।

Example

Node.js API:

HTTP Request
↓
Route
↓
Controller
↓
Service
↓
Repository

↓
Database

Unit test:

Service function

Integration test:

API
↓
Service
↓
Database

একসাথে test করতে পারে।

Integration Test কেন দরকার?

ধরো:

Controller works ✓
Service works ✓
Repository works ✓

তবুও integration fail করতে পারে।

কারণ:

Controller ↔ Service
Service ↔ Repository
Repository ↔ Database

এর মধ্যে contract mismatch থাকতে পারে।

API Integration Test

Example:

POST /users

Test:

```
Send HTTP request
  ↓
Real application stack
  ↓
Test database
  ↓
Response
  ↓
Assert
```

যেমন:

```
const response = await request(app)
  .post("/users")
  .send({
    name: "Rahim",
    email: "rahim@example.com"
  });

expect(response.status).toBe(201);
```

Test Database

Integration testing-এ production database ব্যবহার করবে না।

Better:

```
Production DB
  x

Test DB
  ✓
```

Test database হতে পারে:

Dedicated database
Containerized database
Ephemeral database

Integration Test-এর Scope

সবকিছু real করার দরকার নেই।

উদাহরণ:

API
↓
Service
↓
Real Test DB

কিন্তু external payment provider:

Stripe

এর বদলে test double ব্যবহার করা যেতে পারে।

Mental model:

System under test
↓
Real internal integration
+
Controlled external dependencies

Contract Testing

Microservices বা frontend/backend API architecture-এ useful।

ধরো:

```
Frontend
  ↓
GET /users
  ↓
Backend
```

Frontend expect করে:

```
{
  "id": 1,
  "name": "Rahim"
}
```

Backend accidentally বানালো:

```
{
  "userId": 1,
  "fullName": "Rahim"
}
```

দুটো individually test pass করতে পারে।

কিন্তু contract break করেছে।

Contract testing এই interface verify করতে সাহায্য করে।

Integration Testing Strategy

একটি practical approach:

```
Unit Tests
  ↓
Most tests

Integration Tests
  ↓
Important boundaries

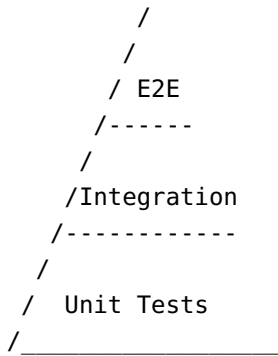
End-to-End Tests
```

↓
Critical user journeys

এটাকে test pyramid-এর সাথে relate করা যায়।

Test Pyramid

Classic model:



মানে সাধারণত:

Many Unit Tests
Some Integration Tests
Few E2E Tests

কারণ E2E tests সাধারণত:

Slower
More expensive
More fragile

কিন্তু Test Pyramid absolute rule নয়

Modern frontend/backend application-এ কখনো **testing trophy** বা অন্য distributions-ও useful হতে পারে।

মূল principle:

Test count নয়, **risk** এবং **confidence** balance করো।

98. Test Doubles — Fake, Stub, Spy, Mock, Dummy

এগুলো খুব ভালোভাবে আলাদা করে বুঝি।

Dummy

Dummy-এর কোনো real behavior দরকার নেই।

শুধু parameter পূরণ করার জন্য দেওয়া হয়।

```
function createUser(name, logger) {  
  // ...  
}
```

Test-এ logger দরকার কিন্তু ব্যবহার হচ্ছে না:

```
const dummyLogger = {};
```

এটা dummy।

Mental model:

```
Need a value  
↓  
Don't care about behavior  
↓  
Dummy
```

Stub

Stub predetermined result দেয়।

```
const userRepository = {  
  findById: async () => ({  
    id: 1,  
    name: "Rahim"  
  })  
}
```

```
}  
};
```

Need controlled answer

↓
Stub

Spy

Spy interaction record করে।

Was called?
How many times?
With what arguments?

Example:

```
emailService.send("hello@example.com");  
  
expect(emailService.send)  
  .toHaveBeenCalledWith("hello@example.com");
```

Mock

Mock expected interaction define করে।

Example concept:

Expected:
repository.save(user)
exactly once

Test-এর মূল concern:

Was expected interaction performed?

Fake

Fake হলো working কিন্তু simplified implementation ।

সবচেয়ে classic example:

```
Real Database
  ↓
Production
In-memory Database
  ↓
Tests
```

যেমন:

```
class FakeUserRepository {
  constructor() {
    this.users = [];
  }

  async save(user) {
    this.users.push(user);
  }

  async findById(id) {
    return this.users.find(user => user.id === id);
  }
}
```

এটা actual database নয়, কিন্তু repository-এর মতো কাজ করছে ।

এটাই fake ।

Fake vs Stub

Stub

একটা predetermined result:

```
findById()  
↓  
always return User
```

Fake

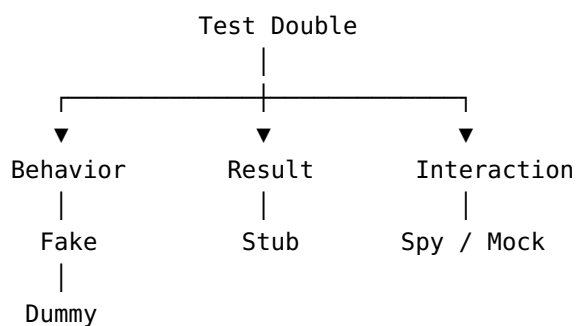
Simplified working implementation:

```
findById()  
↓  
search in memory
```

Complete Comparison

Type	Main Purpose	Example
Dummy	Parameter পূরণ	<code>{}</code>
Stub	Controlled result	<code>findUser() → fake user</code>
Spy	Interaction observe	<code>send called 2 times</code>
Mock	Expected interaction	<code>save() must be called once</code>
Fake	Working simplified implementation	In-memory DB

Test Double Mental Model



আরও সহজভাবে:

Dummy → শুধু দরকার
Stub → answer দেয়
Spy → observe করে
Mock → expectation verify করে
Fake → কাজ করা simplified system

Mocking-এর বড় Danger

সব dependency mock করতে শুরু করলে test বাস্তব application behavior থেকে দূরে যেতে পারে।

ধরো:

```
Service
↓
Repository
↓
Database
```

তুমি সবকিছু mock করলে:

```
Service
↓
Mock Repository
↓
Mock Database
```

Test pass করতে পারে।

কিন্তু real integration broken হতে পারে।

তাই:

Mock only where isolation gives real value.

Over-Mocking

Bad testing:

Implementation details

↓

Mock everything

↓

Brittle tests

যদি code refactor করলেই ২০টা test fail করে কিন্তু user-visible behavior একই থাকে, test suite হয়তো over-coupled।

Behavior vs Implementation

ধরো:

```
function getUsername(user) {  
  return user.name.toUpperCase();  
}
```

Test হওয়া উচিত:

Input user
↓
Expected uppercase name

অতিরিক্তভাবে:

internal helper A called
internal helper B called
variable X used

এসব test করলে implementation-এর সাথে test tightly coupled হয়ে যায়।

Better:

Test public behavior

Deterministic Tests

একটি test একই input-এ একই result দেবে।

Bad:

```
test("works", () => {  
  expect(Math.random()).toBe(...);  
});
```

আরেকটি common problem:

```
Current Date  
Random ID  
Network  
Database state  
Execution order
```

এগুলোর কারণে test flaky হতে পারে।

Flaky Test

একই code:

```
Run 1 → PASS  
Run 2 → FAIL  
Run 3 → PASS
```

এটা flaky test।

Common causes:

```
Race condition  
Timing assumptions  
Real network  
Shared mutable state  
Randomness  
Time dependency  
Test order dependency
```

Test Isolation

একটি test যেন অন্য test-এর state-এর ওপর depend না করে।

Bad:

```
Test A
↓
Global state modified
↓
Test B depends on it
```

Better:

```
Test A → isolated
Test B → isolated
Test C → isolated
```

Test Coverage

Coverage বলতে বোঝাতে পারে:

```
Line coverage
Branch coverage
Function coverage
Statement coverage
```

কিন্তু:

100% coverage $\neq$ 100% bug-free

ধরো:

```
function add(a, b) {
  return a + b;
}
```

100% line coverage পেলেই সব possible behavior tested হয়েছে—এমন নয়।

Coverage useful signal, কিন্তু quality-এর একমাত্র metric নয়।

What Should You Test?

Priority:

```
High business value
+
High risk
+
Complex logic
+
Important boundaries
+
Frequently changing code
```

বিশেষ করে:

```
Authentication
Authorization
Payment
Game rules
Data transformation
Validation
Critical API endpoints
```

এসব জায়গায় strong tests valuable।

Testing Backend Architecture

ধরো Node.js application:

```
Route
↓
Controller
↓
Service
↓
Repository
```

↓
Database

Possible strategy:

Controller

Integration/API test

Service

Unit test

Repository

Integration test

Database

Integration test

Critical user journey

E2E test

এতে প্রতিটি layer-এর জন্য appropriate test type পাওয়া যায়।

Testing Frontend Architecture

React/Next.js application:

Component
↓
Hook
↓
API

Possible strategy:

Pure utility
↓
Unit

```
Component behavior
  ↓
Component/integration test

API boundary
  ↓
Mock server / controlled API

Critical user journey
  ↓
E2E
```

UI implementation detail-এর বদলে user-visible behavior test করা generally বেশি maintainable ।

Testing Workflow

একজন developer feature বানালে:

```
Requirement
  ↓
Expected behavior
  ↓
Unit tests
  ↓
Implementation
  ↓
Integration tests
  ↓
Refactor
  ↓
CI
```

CI pipeline:

```
Push
  ↓
Install dependencies
  ↓
Lint
  ↓
Unit tests
  ↓
```

Integration tests

↓

Build

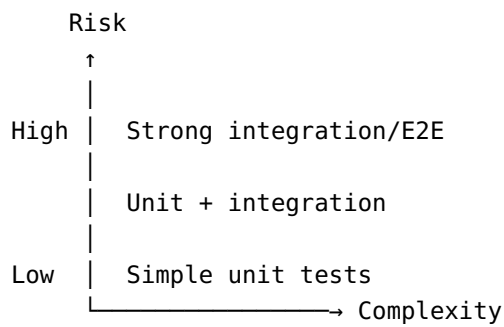
↓

Deploy

Senior Testing Strategy

সব code-এর জন্য একই ধরনের test লিখবে না।

বরং:



Question হবে:

“এই behavior break হলে business-এর কত damage হবে?”

তারপর testing investment নির্ধারণ করবে।

Common Mistakes

✗ শুধু happy path test করা

Valid input

এর পাশাপাশি:

Invalid input
Boundary
Error

Empty
Null
Unexpected state

test করো।

✗ সবকিছু **mock** করা

Mock-heavy tests বাস্তব integration miss করতে পারে।

✗ **Implementation detail test** করা

Refactor করলেই tests break করবে।

✗ **100% coverage obsession**

Coverage target useful হতে পারে, কিন্তু behavior quality বেশি গুরুত্বপূর্ণ।

✗ **Real external services** ব্যবহার করে **unit tests**

Tests slow এবং flaky হতে পারে।

✗ **Tests-এর মধ্যে shared state**

এক test অন্য test-এর ওপর depend করলে suite unreliable হয়।

✗ **Slow test suite ignore** করা

Developer tests run করা বন্ধ করে দিলে test suite-এর value কমে যায়।

Quick Revision

Unit Test
↓
Small isolated behavior

Pure Function

↓

Best place to start

AAA

↓

Arrange → Act → Assert

Stub

↓

Controlled result

Spy

↓

Observe interaction

Mock

↓

Expected interaction

Fake

↓

Working simplified replacement

Dummy

↓

Placeholder value

Property-Based Testing

↓

Test general properties

TDD

↓

Red → Green → Refactor

Integration Test

↓

Multiple real components working together

Test Pyramid

↓

Many unit + some integration + few E2E

Interview Questions

Beginner

1. Unit testing কী?
2. Pure function testing-এর জন্য ভালো কেন?
3. AAA pattern কী?
4. Stub কী?
5. Mock কী?
6. Spy কী?
7. Fake কী?
8. Dummy কী?

Intermediate

1. Stub আর Mock-এর পার্থক্য কী?
2. Stub আর Fake-এর পার্থক্য কী?
3. Unit এবং Integration test-এর পার্থক্য কী?
4. Property-based testing কী?
5. TDD-এর Red/Green/Refactor cycle কী?
6. Test isolation কেন দরকার?
7. Flaky test কী?
8. Code coverage কী?

Senior

1. কখন mock করবে আর কখন real dependency ব্যবহার করবে?
2. Over-mocking কী এবং কেন dangerous?
3. Behavior testing এবং implementation-detail testing-এর পার্থক্য কী?
4. Node.js API-এর জন্য unit/integration test boundary কীভাবে design করবে?
5. Test pyramid কেন useful?
6. 100% test coverage থাকলেও application-এ bug থাকতে পারে কেন?
7. Property-based testing কোন ধরনের system-এ সবচেয়ে useful?
8. External payment API test করার architecture কী হবে?
9. Database repository-এর জন্য unit test নাকি integration test—কখন কোনটা?
10. একটি large Node.js application-এর complete testing strategy কীভাবে design করবে?

Practice Tasks

Task 1 — Pure Function Tests

এই functions বানাও:

```
add(a, b)
subtract(a, b)
calculateDiscount(price, percentage)
isAdult(age)
```

প্রতিটির:

```
Happy path
Boundary
Invalid input
```

test লেখো।

Task 2 — Stub

একটি:

```
UserService
```

বানাও।

যেটা:

```
UserRepository.findById()
```

call করে।

Real database-এর বদলে stub দিয়ে predetermined user return করাও।

Task 3 — Spy

একটি:

```
EmailService
```

বানাও।

User registration-এর পরে:

```
emailService.sendWelcomeEmail()
```

call হয়েছে কিনা এবং সঠিক user-এর জন্য call হয়েছে কিনা verify করে।

Task 4 — Fake Repository

একটি:

```
FakeUserRepository
```

বানাও যেখানে data memory-তে থাকবে।

Methods:

```
save()  
findById()  
delete()
```

তারপর `UserService` -এর test-এ real database-এর পরিবর্তে fake repository ব্যবহার করে।

Task 5 — Property-Based Thinking

এই function:

```
function reverse(str) {  
  return str.split("").reverse().join("");  
}
```

এর অন্তত ৩টি property বের করে।

Hint:

```
reverse(reverse(x)) === x
```

আরও:

```
length unchanged
```

Task 6 — TDD

একটি:

```
calculateShippingCost()
```

function TDD দিয়ে বানাও।

Rules:

```
Weight <= 1kg → 50  
Weight <= 5kg → 100  
Weight > 5kg → 200
```

Process:

```
RED  
↓  
GREEN  
↓  
REFACTOR
```

প্রতিটি step আলাদা commit করেও practice করতে পারো।

Task 7 — API Integration Test

একটি:

```
POST /users
```

endpoint বানাও।

Test করে:

Valid request → 201

Invalid request → 400

Duplicate email → appropriate error

Missing required field → 400

এখানে test database ব্যবহার করো।

Task 8 — Test Strategy Design

তোমার একটি application আছে:

Login
User Profile
Payment
Game
Chat
Notification

প্রতিটি feature-এর জন্য decide করো:

Unit?
Integration?
E2E?

এবং কেন সেটা বেছে নিয়েছো explain করো।

Mastery Checklist

- ☐ Unit testing কী
- ☐ Pure function কেন easy to test
- ☐ AAA pattern
- ☐ Happy path এবং edge cases
- ☐ Test isolation
- ☐ Deterministic test
- ☐ Flaky test
- ☐ Stub
- ☐ Spy

- [] Mock
- [] Fake
- [] Dummy
- [] Test double
- [] Mock vs Stub
- [] Stub vs Fake
- [] Property-based testing
- [] Property বনাম example
- [] TDD
- [] Red / Green / Refactor
- [] Behavior vs implementation testing
- [] Integration testing
- [] API integration test
- [] Test database
- [] Contract testing
- [] Test pyramid
- [] Coverage
- [] Over-mocking
- [] CI testing strategy

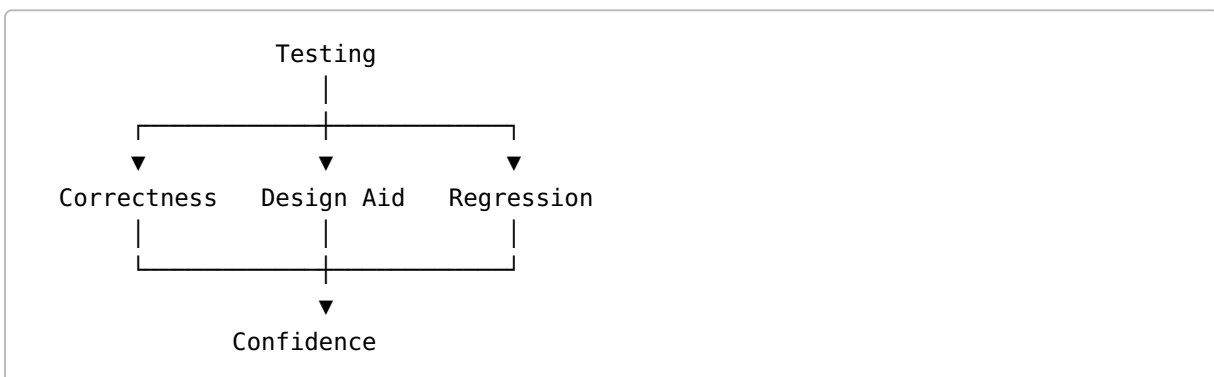
Final Mental Model

Testing-কে শুধু:

"Bug ধরার tool"

হিসেবে দেখবে না।

বরং:



আর test type-এর mental model:

Pure Logic
↓
Unit Test

Component Boundary
↓
Integration Test

System Boundary
↓
Integration / Contract Test

Critical User Journey
↓
E2E Test

Test doubles:

Dummy → শুধু parameter পূরণ

Stub → controlled answer

Spy → interaction observe

Mock → expected interaction verify

Fake → simplified but working implementation

সবশেষে সবচেয়ে important principle:

“Test everything” ভালো **testing strategy** নয়; **“test the right behavior at the right boundary”**
হলো ভালো **testing strategy**।

আর senior developer-এর testing mindset:

What can break?
↓
How costly is it?
↓
Where is the boundary?
↓
What is the cheapest reliable test?

↓
How much confidence do I get?

এভাবেই testing-কে code-এর পরে একটা extra কাজ না বানিয়ে **development process**-এর অংশ বানানো যায়।